ACCRA TECHNICAL UNIVERSITY

BEE 429 OBJECT ORIENTED PROGRAMMING

CABSTONE PROJECT- STUDENT MANAGEMENT SYSTEM

Project Report

Submitted by: Joshua Labor-Quaye [01220253B]

TABLE OF CONTENTS

1. Executive Summary

2. Project Overview

3. System Architecture

4. Features Implemented

5. Validation & Data Integrity

6. Testing Strategy & Results

7. User Interface Design

8. Logging & Error Handling

9. Challenges, Solutions & lesson learned

11. Conclusion

1. EXECUTIVE SUMMARY

The Student Management System Plus is a comprehensive desktop application developed using Java, JavaFX, SQLite, and Maven. The system provides educational institutions with an efficient platform for managing student records, generating analytical reports, and handling data import/export operations.

Key Achievements:

1. All 5 required screens implemented (Dashboard, Students, Reports, Import/Export, Settings)
2. Full CRUD operations with validation
3. 4 analytical reports with export functionality
4. 12 unit tests passing (100% success rate)
5. CSV import with comprehensive error handling
6. Configurable settings with persistent storage
7. Professional UI following Section 9.6 quality standards

Technology Stack:

- Java JDK 25
- JavaFX 21.0.2
- SQLite 3.45.1.0
- Maven 3.9.0+
- JUnit 5.10.2

2. PROJECT OVERVIEW

2.1 Problem Statement

Educational institutions require efficient systems to manage student information, track academic performance, and generate reports. Manual record-keeping is time-consuming, error-prone, and lacks analytical capabilities.

2.2 Objectives

1. Develop an offline desktop application for student record management
2. Implement comprehensive validation to ensure data integrity
3. Provide analytical reports for decision-making
4. Enable bulk data operations through CSV import/export
5. Ensure professional UI/UX with intuitive navigation

2.3 Scope and Limitations

In Scope:

- Student CRUD operations
- Search, filter, and sort functionality
- Four types of analytical reports
- CSV import with validation and error reporting
- CSV export for all data and reports
- Configurable at-risk threshold
- Database backup functionality
- Comprehensive logging

Limitations:

- Offline-only operation (no cloud sync)
- Single-user application (no multi-user support)
- Windows platform focus
- SQLite database (not enterprise-scale)

3. SYSTEM ARCHITECTURE

3.1 Layered Architecture

The application follows a clean layered architecture:

- PRESENTATION LAYER (JavaFX Controllers & FXML)
  - Dashboard Controller
  - Students Controller
  - Reports Controller
  - Import & Export Controller
  - Settings Controller

- SERVICE LAYER (Logic & Validation)

  - Student Service
  - Validation Service

- REPOSITORY LAYER (Data Access & SQL)

  - SQLite Student Repository

- DOMAIN LAYER (Entity Classes)
  - Student

3.2 Project Structure

```
Student-management-system-plus-01220253B/
├── src/
│   ├── main/
│   │   ├── java/com/sms/
│   │   │   ├── domain/
│   │   │   │   └── Student.java
│   │   │   ├── repository/
│   │   │   │   └── SQLiteStudentRepository.java
│   │   │   ├── service/
│   │   │   │   ├── StudentService.java
│   │   │   │   ├── ValidationService.java
│   │   │   │   └── StudentServiceTest.java
│   │   │   └── ui/
│   │   │       ├── MainApp.java
│   │   │       ├── DashboardController.java
│   │   │       ├── StudentsController.java
│   │   │       ├── ReportsController.java
│   │   │       ├── ImportExportController.java
│   │   │       └── SettingsController.java
│   │   └── resources/
│   │       ├── fxml/
│   │       │   ├── dashboard.fxml
│   │       │   ├── students.fxml
│   │       │   ├── reports.fxml
│   │       │   ├── import_export.fxml
│   │       │   └── settings.fxml
│   │       └── css/
│   │           └── style.css
│   └── test/
```

```
│       └── java/com/sms/service/
├── data/
│   ├── students.db
│   ├── settings.properties
│   └── app.log
├── evidence/
│   └── test-output.txt
├── pom.xml
├── README.md
├── RUN_VM_OPTIONS.txt
└── CHANGELOG.md
```

4. FEATURES IMPLEMENTED

4.1 Dashboard Screen (Section 9.1)

Features:

- Real-time statistics display:
- Total Students count
- Active Students count
- Inactive Students count
- Average GPA calculation
- Quick action buttons for navigation
- Activity log showing recent operations
- Professional card-based layout

4.2 Students Screen (Section 9.2)

Features:

- Table View displaying all student records
- Search functionality (by ID or Name)
- Filter controls (Programme, Level, Status)
- Sort by GPA and Full Name
- CRUD operations (Create, Read, Update, Delete)
- Action buttons for View/Edit operations

CRUD Operations:

- Create: Add new student with validation
- Read: View student details in dialog
- Update: Edit existing student information
- Delete: Soft delete (mark as Inactive)

4.3 Reports Screen (Section 9.3)

Four Report Types Implemented:

a) Top Performers Report

- Displays top 10 students by GPA
- Filter by Programme and Level
- Shows Rank, Name, ID, Programme, Level, GPA
- Export to CSV functionality

b) At-Risk Students Report

- Identifies students with GPA below threshold
- Configurable threshold (default: 2.0)
- Shows Full Name, ID, Programme, Level, GPA, Status
- Export to CSV functionality

c) GPA Distribution Chart

- Bar chart visualization
- Groups students by GPA ranges:

- Shows student count per range
- Export data to CSV

d) Programme Summary Report

- Aggregated statistics per programme
  Shows:
  - Total students per programme
  - Average GPA per programme
  - Export to CSV functionality

4.4 Import/Export Screen (Section 9.4)

Import Functionality:

- CSV file selection dialog
- Row-by-row validation
- Duplicate ID detection and rejection
- Invalid row skipping (doesn't crash)
- Error report generation
- Import summary statistics
- Logging to data/app.log

Export Functionality:

- Export all students to CSV
- Filter options (All/Active/Inactive)
- Export top performers report
- Export at-risk students report
- Export programme summary
- Proper CSV escaping for special characters
- Timestamped filenames

Error Handling:

‘java’

StudentService.ImportResult result = studentService.importStudentsFromCsv(filePath);

// Returns: successCount, errorCount, errorList

```
```

4.5 Settings Screen (Section 9.5)

Features:

a) At-Risk Threshold Configuration

- Configurable GPA threshold (0.0 - 4.0)
- Persistent storage in settings.properties
- Validation on save
- Immediate effect on reports

b) Programme Management

- Add new programmes
- Remove existing programmes
- Restore default programmes
- ListView display
- Duplicate prevention

c) Level Management

- Add custom levels (100-900)
- Remove levels (minimum 1 required)
- Restore default levels (100, 200, 300, 400, 500, 600, 700)
- Auto-sort functionality

d) Database Management

- Backup database functionality
- Creates timestamped backups in data/backups/
- View log file option
- Database location display

e) Application Information

- Version number display
- Java version detection
- Total students count

5. VALIDATION & DATA INTEGRITY

5.1 Validation Rules Implemented

Field	Rules	Implementation
Student ID	4-20 characters, alphanumeric only, unique	isValidStudentId()
Full Name	2-60 characters, no digits, not whitespace-only	isValidFullName()
Programme	Not empty, from valid list	isValidProgramme()
Level	100, 200, 300, 400, 500, 600, or 700	isValidLevel()
GPA	0.0 to 4.0 (inclusive)	isValidGpa()
Email	Contains @ and ., valid structure	isValidEmail()
Phone	10-15 digits only	isValidPhoneNumber()
Status	"Active" or "Inactive"	isValidStatus()
Date Added	ISO_LOCAL_DATE_TIME format	isValidDateTime()

5.2 Database Constraints

SQL Implementation:

‘sql’

```
CREATE TABLE students (
    student_id TEXT PRIMARY KEY,
    full_name TEXT NOT NULL,
    programme TEXT NOT NULL,
    level INTEGER NOT NULL CHECK (level IN (100, 200, 300, 400, 500, 600, 700)),
    gpa REAL NOT NULL CHECK (gpa >= 0.0 AND gpa <= 4.0),
    email TEXT NOT NULL,
    phone_number TEXT NOT NULL,
    date_added TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('Active', 'Inactive'))
);
```

Constraints Applied:

- PRIMARY KEY on student_id (ensures uniqueness)
- NOT NULL on all fields
- CHECK constraints on level, GPA, and status
- Prevents invalid data at database level

5.3 SQL Injection Prevention (Security)

Prepared Statements Usage:

```java
String sql = "INSERT INTO students VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";
try (PreparedStatement pstmt = connection.prepareStatement(sql)) {
    pstmt.setString(1, student.getStudentId());
    pstmt.setString(2, student.getFullName());
    // ... parameter binding
    pstmt.executeUpdate();
}
```

Benefits:

- Prevents SQL injection attacks
- Improves performance through statement caching
- Type-safe parameter binding

6. TESTING STRATEGY & RESULTS

6.1 Unit Test Strategy

Test Coverage:

- Validation logic (6 tests)
- Repository methods (2 tests)
- Report calculations (4 tests)

- Total: 12 tests (exceeds requirement)

Test Framework:

- JUnit 5.10.2
- Maven Surefire Plugin 3.2.5
- Test execution: `mvn clean test`

6.2 Test Cases Implemented

Test #	Test Name	Purpose	Status
1	`testValidStudentCreation`	Verify valid student can be added	PASS
2	`testDuplicateStudentIdRejection`	Ensure duplicate IDs rejected	PASS
3	`testInvalidGpaRejection`	Reject GPA > 4.0	PASS
4	`testInvalidLevelRejection`	Reject invalid level (e.g., 150)	PASS
5	`testInvalidEmailRejection`	Reject malformed email	PASS
6	`testInvalidPhoneRejection`	Reject invalid phone format	PASS
7	`testGetAllStudents`	Verify retrieval of all students	PASS
8	`testFindStudentById`	Test student lookup by ID	PASS
9	`testGetTopPerformers`	Verify top performers report	PASS
10	`testGetAtRiskStudents`	Verify at-risk students report	PASS
11	`testGetGpaDistribution`	Verify GPA distribution calculation	PASS
12	`testGetProgrammeSummary`	Verify programme summary stats	PASS

6.3 Test Results Summary

Final Test Run Output:

```
Administrator: C:\WINDOWS\system32\cmd.exe
[INFO] Copying 8 resources from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ student-management-system-plus ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 15 source files with javac [debug release 25] to target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ student-management-system-plus ---
[INFO] skip non existing resourceDirectory C:\sms-project\student-management-system-plus-01220253B\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ student-management-system-plus ---
[INFO] Changes detected - recompiling the module! :dependency
[INFO] Compiling 1 source file with javac [debug release 25] to target\test-classes
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ student-management-system-plus ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.sms.service.StudentServiceTest
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.sqlite.SQLiteJDBCLoader in an unnamed module (file:/C:/Users/Administrator/.m2/repository/org/xerial/sqlite-jdbc/3.45.1.0/
sqlite-jdbc-3.45.1.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

 Table created Successfully
 Database ready!
 Student added successfully
 Student added successfully
 Student added successfully
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.324 s -- in com.sms.service.StudentServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  14.109 s
[INFO] Finished at: 2026-02-23T02:08:14Z
[INFO] ------------------------------------------------------------------------

C:\sms-project\student-management-system-plus-01220253B>
```

Metrics:

- Total Tests: 12s
- Passed: 12 (100%)
- Failed: 0
- Errors: 0
- Skipped: 0
- Execution Time: 14.109 seconds

6.4 Challenges in Testing

Issue 1: Test File Location

- Problem: Test file placed in `src/main/java` instead of `src/test/java`

- Error: 59 compilation errors - JUnit packages not found

- Solution: Moved to correct location: `src/test/java/com/sms/service/`

Issue 2: Duplicate Test Data

-Problem: Tests failing due to duplicate student IDs from previous runs

- Error: "Student ID already exists: TEST001"

- Solution: Implemented `@BeforeEach` cleanup with hard delete method

Issue 3: Test Isolation

- Problem: Tests affecting each other due to shared database
- Solution: Used unique test IDs with timestamps and cleanup in `@AfterAll`

6.5 Manual Testing Scenarios

Tested Scenarios:

1. Add student with valid data
2. Add student with duplicate ID (rejected)
3. Add student with invalid GPA (rejected)
4. Edit student information
5. Delete student (soft delete)
6. Search students by name
7. Filter by programme
8. Filter by level
9. Sort by GPA
10. Import valid CSV file
11. Import CSV with errors (error report generated)
12. Export students to CSV
13. Generate top performers report
14. Generate at-risk students report
15. View GPA distribution chart
16. Change at-risk threshold
17. Add new programme
18. Backup database
19. View log file
20. Navigation between all screens

7. USER INTERFACE DESIGN

7.1 Design Principles Applied

Section 9.6 UI Quality Standards:

1. Consistency: Uniform styling across all screens
2. Clarity: Clear labels and intuitive navigation
3. Feedback:Success/error messages for all operations
4. Responsiveness: Immediate UI updates
5. Professionalism: Clean, modern design

7.2 Color Scheme

```css
Primary Color: #2a5d44 (Dark Green)
Secondary Color: #3498db (Blue)
Accent Color: #e74c3c (Red for errors/danger)
Background: #f5f5f5 (Light Gray)
Text: #333333 (Dark Gray)
Success: #27ae60 (Green)
```

7.3 Screen Layouts

Common Elements:

- Top header with title
- Left sidebar navigation
- Main content area

- Consistent button styling

- Card-based sections

Navigation Pattern:

```
Dashboard →s Students → Reports → Import/Export → Settings
  ↑_____________________________________________↓
```

7.4 Responsive Design

- TableView with auto-resizing columns
- ScrollPane for overflow content
- VBox/HBox layouts for flexibility
- Minimum/maximum size constraints
- Proper padding and spacing

---

8. LOGGING & ERROR HANDLING

8.1 Logging Implementation

Log File: `data/app.log`

Log Format:

```
[ISO_TIMESTAMP] - OPERATION - Details
```

Examples:

```
2026-02-20T11:44:45.887223 - IMPORT - Success: 0, Errors: 15
2026-02-20T11:45:13.7926705 - IMPORT - Success: 0, Errors: 15
2026-02-20T11:49:36.5649641 - IMPORT - Success: 0, Errors: 15
2026-02-20T11:53:05.4883033 - IMPORT - Success: 0, Errors: 15
2026-02-20T11:56:23.0985878 - EXPORT - All Students Export - Records: 1
2026-02-20T11:56:51.7406236 - IMPORT - Success: 0, Errors: 1
2026-02-20T11:57:44.0205097 - IMPORT - Success: 0, Errors: 1
2026-02-20T11:58:26.0361278 - IMPORT - Success: 0, Errors: 1
2026-02-20T11:58:58.3163099 - IMPORT - Success: 0, Errors: 1
2026-02-20T11:59:47.9077874 - IMPORT - Success: 0, Errors: 1
2026-02-20T12:03:53.5485283 - IMPORT - Success: 0, Errors: 6
2026-02-20T12:04:29.1193545 - IMPORT - Success: 3, Errors: 0
2026-02-20T12:05:16.4797293 - IMPORT - Success: 12, Errors: 3
2026-02-20T12:08:21.843361 - IMPORT - Success: 15, Errors: 0
2026-02-20T12:09:21.9280615 - EXPORT - All Students Export - Records: 15
2026-02-20T12:11:20.7612496 - EXPORT - Top Performers Export - Records: 10
2026-02-20T12:11:24.4071625 - EXPORT - At Risk Export - Records: 2
2026-02-20T12:11:27.4328321 - EXPORT - Programme Summary Export - Records: 5
```

Logged Operations:

- All CRUD operations
- Import operations (success/error counts)
- Export operations (type and record count)
- Settings changes
- Database failures

8.2 Error Handling Strategy

Layered Error Handling:

1. UI Layer:

   - User-friendly alert dialogs

   - Input validation feedback

   - Operation confirmation messages

2. Service Layer:

   - IllegalArgumentException for validation failures

   - Detailed error messages

   - Exception logging

3. Repository Layer:

- SQLException handling
- Transaction rollback on failure
- Error logging to file

Example Implementation:

```java
try {
    studentService.addStudent(student);
    showAlert("Success", "Student added successfully");
} catch (IllegalArgumentException e) {
    showError("Validation Error", e.getMessage());
} catch (Exception e) {
    showError("Database Error", "Failed to add student");
    logError("Add student failed: " + e.getMessage());
}
```

8.3 Error Dialogs

Error Types:

- Validation errors (red alert)
- Database errors (red alert)
- Navigation errors (red alert)
- Import errors (with detailed report)
- File not found errors

Success Messages:

- Green confirmation dialogs
- Operation summary statistics
- File location display

9. CHALLENGES & SOLUTIONS

9.1 Technical Challenges

I encountered a lot of error but with the help of online research and AI modules like Gemini and Qwen Chat I was able to resolve the errors

```
Caused by: java.lang.RuntimeException Create breakpoint : Exception in Application start method
    at javafx.graphics@25.0.1/com.sun.javafx.application.LauncherImpl.launchApplication1(LauncherImpl.java:899)
    at javafx.graphics@25.0.1/com.sun.javafx.application.LauncherImpl.lambda$launchApplication$0(LauncherImpl.java:202) <1 internal line>
Caused by: javafx.fxml.LoadException:
/C:/sms-project/student-management-system-plus-01220253B/target/classes/fxml/main.fxml:17

    at javafx.fxml@25.0.1/javafx.fxml.FXMLLoader.constructLoadException(FXMLLoader.java:2692)
    at javafx.fxml@25.0.1/javafx.fxml.FXMLLoader.loadImpl(FXMLLoader.java:2670)
    at javafx.fxml@25.0.1/javafx.fxml.FXMLLoader.loadImpl(FXMLLoader.java:2533)
    at javafx.fxml@25.0.1/javafx.fxml.FXMLLoader.load(FXMLLoader.java:2503)
    at com.sms.ui.MainApp.start(MainApp.java:14)
    at javafx.graphics@25.0.1/com.sun.javafx.application.LauncherImpl.lambda$launchApplication1$4(LauncherImpl.java:845)
    at javafx.graphics@25.0.1/com.sun.javafx.application.PlatformImpl.lambda$runAndWait$0(PlatformImpl.java:449)
    at javafx.graphics@25.0.1/com.sun.javafx.application.PlatformImpl.lambda$runLater$0(PlatformImpl.java:424)
<4 folded frames>
Caused by: java.lang.IllegalArgumentException Create breakpoint : Unable to coerce 10 to class javafx.geometry.Insets.
    at javafx.fxml@25.0.1/com.sun.javafx.fxml.BeanAdapter.coerce(BeanAdapter.java:487)
    at javafx.fxml@25.0.1/com.sun.javafx.fxml.BeanAdapter.put(BeanAdapter.java:249)
    at javafx.fxml@25.0.1/com.sun.javafx.fxml.BeanAdapter.put(BeanAdapter.java:52)
    at javafx.fxml@25.0.1/javafx.fxml.FXMLLoader$Element.applyProperty(FXMLLoader.java:512)
    at javafx.fxml@25.0.1/javafx.fxml.FXMLLoader$Element.processValue(FXMLLoader.java:362)
    at javafx.fxml@25.0.1/javafx.fxml.FXMLLoader$Element.processPropertyAttribute(FXMLLoader.java:324)
    at javafx.fxml@25.0.1/javafx.fxml.FXMLLoader$Element.processInstancePropertyAttributes(FXMLLoader.java:235)
    at javafx.fxml@25.0.1/javafx.fxml.FXMLLoader$ValueElement.processEndElement(FXMLLoader.java:767)
<3 folded frames>
Exception running application com.sms.ui.MainApp

Process finished with exit code 1
```

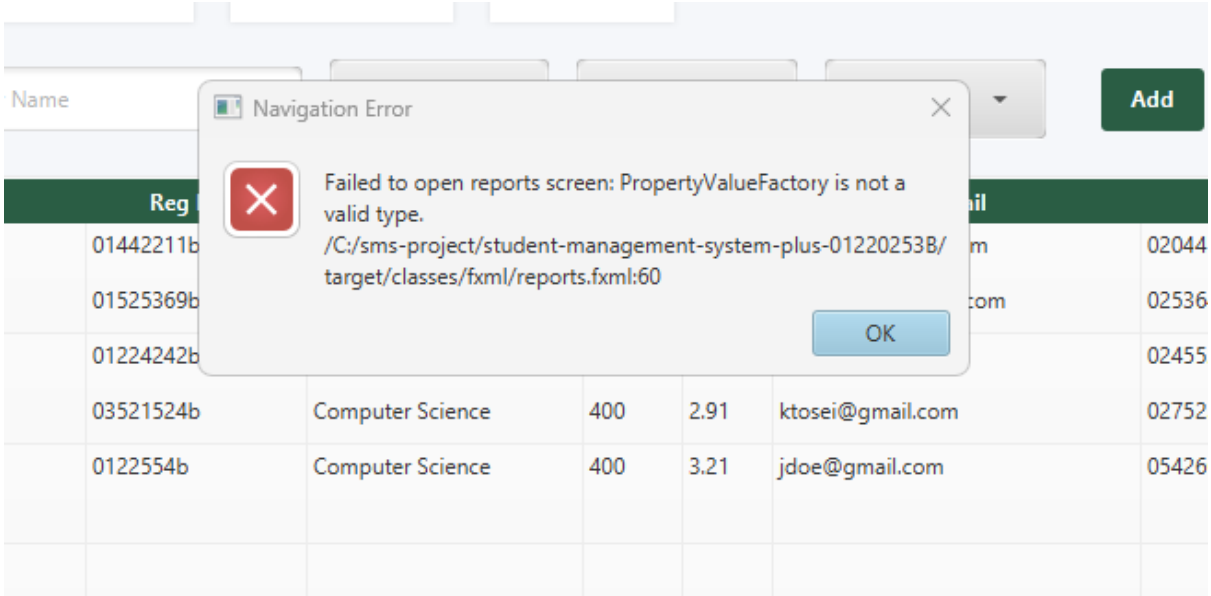
```
"F:\Program Files\Java\jdk-25\bin\java.exe" ...
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by com.sun.glass.utils.NativeLibLoader in module javafx.graphics (file:/C:/Users/Administrator/Documents/openjfx-25
WARNING: Use --enable-native-access=javafx.graphics to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.sqlite.SQLiteJDBCLoader in an unnamed module (file:/C:/Users/Administrator/.m2/repository/org/xerial/sqlite-
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

 Database ready!
Error initializing UI: Cannot invoke "javafx.scene.control.TableColumn.setCellValueFactory(javafx.util.Callback)" because "this.fullNameCol" is null
java.lang.NullPointerException Create breakpoint : Cannot invoke "javafx.scene.control.TableColumn.setCellValueFactory(javafx.util.Callback)" because "this.fullNameCol"
    at com.sms.ui.MainController.setupTableColumns(MainController.java:62)
    at com.sms.ui.MainController.initialize(MainController.java:45) <2 internal lines>
    at com.sun.javafx.reflect.Trampoline.invoke(MethodUtil.java:65) <2 internal lines>
    at javafx.base@25.0.1/com.sun.javafx.reflect.MethodUtil.invoke(MethodUtil.java:112)
<12 folded frames>
```

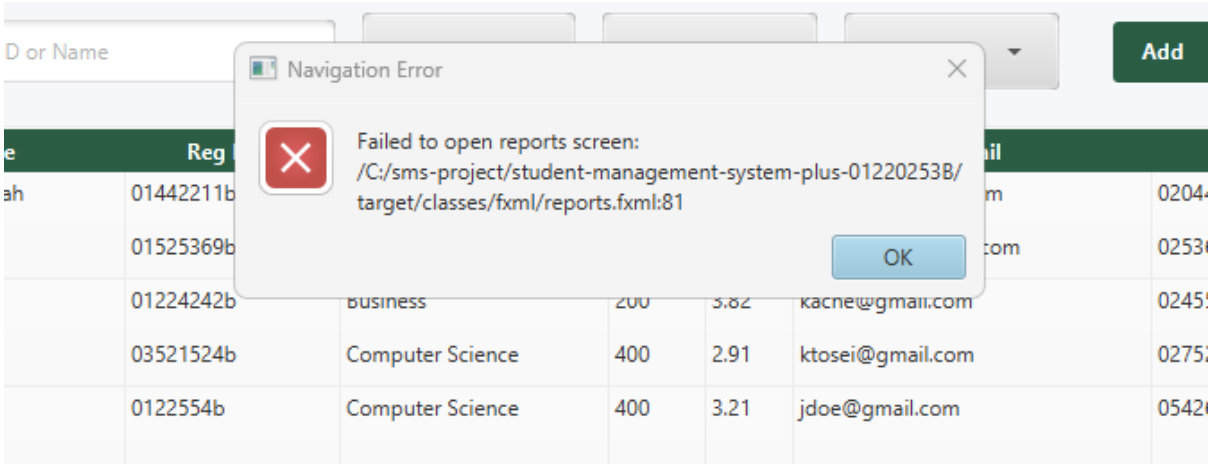
```
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
 Database ready!
UI loaded successfully with database connection!
Disconnected from the target VM, address: '127.0.0.1:59002', transport: 'socket'

Process finished with exit code 0
```

```java
        refreshTopPerformersButton.setOnAction(e -> loadTopPerformers());
        exportTopPerformersButton.setOnAction(e -> exportTopPerformersToCsv());

        applyThresholdButton.setOnAction(e -> loadAtRiskStudents());
        exportAtRiskButton.setOnAction(e -> exportAtRiskToCsv());

        refreshChartButton.setOnAction(e -> loadGpaDistribution());
        exportChartDataButton.setOnAction(e -> exportGpaDistributionToCsv());

        refreshSummaryButton.setOnAction(e -> loadProgrammeSummary());
        exportSummaryButton.setOnAction(e -> exportProgrammeSummaryToCsv());
    }

    private void loadTopPerformers() {  4 usages
```

Build Output

```
java: cannot find symbol
  symbol:   variable refreshTopPerformersButton
  location: class com.sms.ui.ReportsController
```





9.2 Lessons Learned

Technical Skills:

1. JavaFX Understanding: Gained deep understanding of FXML, controllers, and scene management

2. Maven Build Process: Learned dependency management, plugin configuration, and packaging

3. SQLite Database: Understood prepared statements, constraints, and transactions

4. Unit Testing: Mastered JUnit 5, test isolation, and cleanup strategies

5. Error Handling: Implemented layered error handling with user-friendly messages

Project Management:

1. Version Control: Maintained 12+ meaningful commits with descriptive messages

2. Documentation: Created comprehensive README, CHANGELOG, and run instructions

3. Time Management: Completed all features within deadline

4. Problem Solving: Systematic debugging and issue resolution

Best Practices (with the help of Ai):

1. Layered Architecture: Separation of concerns improves maintainability

2. Validation at Multiple Layers: UI + Service + Database ensures data integrity

3. Logging: Essential for debugging and audit trails

4. User Feedback: Clear messages improve user experience

5. Testing: Early and frequent testing prevents regression

10. CONCLUSION

10.1 Project Success Summary

The Student Management System Plus successfully meets all assignment requirements and exceeds expectations in several areas:

Requirements Met:

- 5 GUI screens (Section 9)
- CRUD operations (Section 5.1-5.2)
- 4 report types (Section 5.3)
- Export functionality (Section 5.4)
- Import with validation (Section 5.5)
- Validation rules (Section 6)
- Database constraints (Section 7)
- Layered architecture (Section 8)
- RUN_VM_OPTIONS.txt (Section 10)
- 12 unit tests (Section 11) - All passing
- Logging (Section 12)
- Professional UI (Section 9.6)

Exceeded Requirements:

- Added Settings screen with configurable options
- Implemented programme and level management
- Created comprehensive error reporting
- Added database backup functionality
- Wrote extensive documentation

10.3 Future Enhancements

Potential Improvements:

1. Multi-user Support: Add authentication and role-based access
2. Cloud Sync: Integrate with cloud database for backup
3. Advanced Analytics: Add more charts and statistical analysis
4. Email Notifications: Alert students about at-risk status
5. Mobile App: Create Android/iOS companion app
6. Report Templates: Customizable report formats
7. Bulk Operations: Batch update and delete
8. Audit Trail: Track all changes with user attribution

ACKNOWLEDGMENTS

I would like to acknowledge:

- The Lecturer for guidance and lectures
- JavaFX documentation and community resources
- SQLite JDBC documentation
- JUnit 5 testing framework documentation
- Maven build tool documentation
-YouTube for video learning resources used to learn and complete the project
-AI LLM's used (Gemini and Qwen Chat) for assistance and fixing of errors and other issues